Predicting Sediment Particle Size Distributions Using Machine Learning

A Random Forest and Support Vector Regression replication of Egan (Stanford CS229)

Siddhesh Dattatray Koditkar (PES2UG24CS502) · Sumedh B Rao (PES2UG24CS535)

PES University · UE24CS352A – Machine Learning · Mini-Project Write-up · October 2026

Abstract. Suspended marine sediment governs coastal erosion, contaminant transport and carbon sequestration, and its behaviour depends on the particle size distribution (PSD). We replicate the approach of Egan [1] to predict the median particle diameter d_{50} and the PSD variance σ^2 from eight physical, optical and biological measurements. Because the original South San Francisco Bay field data are not public, we generate a physically motivated synthetic dataset of 1,648 hourly samples with nonlinear interactions, noise, missing values and outliers. On a held-out 30% test set, a Random Forest (400 trees, all 8 features) attains $R^2 = 0.840$ for d_{50} and 0.915 for σ^2, while an RBF-kernel Support Vector Regressor using only four features attains 0.842 and 0.903. Both exceed the $R^2 > 0.75$ level reported in the original paper.

Keywords: random forest · support vector regression · flocculation · particle size distribution · feature importance · out-of-bag error

1 Problem Statement

Particle settling velocity is the most critical, yet least known, parameter in numerical sediment-transport models. It can be related to particle size, but size varies by orders of magnitude within seconds through flocculation (aggregation and break-up), and no general semi-empirical model predicts it accurately. We treat this as a supervised regression task: given the eight measurable variables $\mathbf{x} = (S, u_b, n_p, T, a_{676}/a_{650}, a_{450}/a_{676}, \text{chl-}a, u)$, learn two mappings $\mathbf{x} \to d_{50}$ (median particle diameter) and $\mathbf{x} \to \sigma^2$ (PSD variance). Success is measured by the coefficient of determination R^2 between predicted and true values on unseen data (Eq. 6).

2 Dataset

The paper's data (three one-month LISST-100x / ac-9 campaigns, 1,648 hourly examples) are not available online, so `Dataset_Generator.py` synthesises a dataset of identical size and structure (n = 1,648, hourly from 2018-07-01, seed 42). Inputs follow the distributions in Table 1; the two targets are generated from nonlinear relationships that include a salinity–wave interaction, a biological (chlorophyll) term and a spectral term:

$$d_{50} = 5S + 300u_b + 90n_p + 2T + 30z_S(u_b - \bar{u}_b) + 12\ln(1 + \text{chl-}a) + 40(a_{676}/a_{650} - 1)^2 + \eta_d \tag{1}$$

$$\sigma^2 = 0.12\, d_{50}{}^{1.25} + 8u + 20(a_{450}/a_{676} - 0.9) + \eta_\sigma \tag{2}$$

where z_S is standardised salinity, η denotes heteroscedastic Gaussian noise (≈ 3% and 2% of the signal), and d_{50} inside Eq. (2) is the noise-free value from Eq. (1). We then inject 2% missing feature values and 0.5% target outliers (1.5–3× inflation) so that the preprocessing stage is meaningful.

Table 1. Input features and targets of the simulated dataset (t = hour index).

Symbol	Meaning	Simulated distribution	Used by
S	Salinity	Uniform(20, 35)	RF, SVR
u_b	Wave-orbital velocity	AR(1): $\mu = 0.25$, $\varphi = 0.7$, clipped to [0.01, 1]	RF, SVR
n_p	Particle index of refraction	Normal(1.38, 0.03²)	RF, SVR
T	Water temperature (°C)	16 + 6 sin(2πt/360) + Normal(0, 1.2²)	RF, SVR
a_{676}/a_{650}	Algal (organic) absorption peak	Normal(1.0, 0.18²), clipped [0.4, 1.8]	RF
a_{450}/a_{676}	Detrital (inorganic) peak	Normal(0.9, 0.2²), clipped [0.3, 1.6]	RF
chl-a	Chlorophyll-a (organic proxy)	Lognormal (median 0.8, s = 0.9)	RF
u	Mean tidal / flow velocity	AR(1): $\mu = 0.18$, $\varphi = 0.6$, clipped [0.01, 0.8]	RF
d_{50}, σ^2	Targets (median diameter, variance)	Eqs. (1) and (2)	–

Preprocessing (`Models.py`): drop the timestamp; remove rows with missing values (1,648 → 1,402); discard rows above the 99th percentile of either target, which removes the injected outliers (→ 1,373 rows); then a random 70/30 split gives 961 training and 412 test samples (`random_state = 42`).

3 Methodology

3.1 Random Forest regression

A Random Forest [2] averages B decision trees T_b, each grown on a bootstrap sample of the training data with random feature subsets at each split:

$$\hat{y}(\mathbf{x}) = (1/B)\, \Sigma_{b=1}{}^{B}\, T_b(\mathbf{x}) \tag{3}$$

Samples left out of each bootstrap give the out-of-bag (OOB) score, an internal unbiased estimate used to choose B. The OOB score saturates by roughly 60 trees (Fig. 2b); we use B = 400 with all eight features. Feature importance (mean impurity decrease) is shown in Fig. 2a.

3.2 Support Vector Regression

ε-SVR fits $f(\mathbf{x}) = \Sigma_i(\alpha_i - \alpha_i^*)K(\mathbf{x}_i, \mathbf{x}) + b$ by solving the convex problem

$$\min_{\mathbf{w},b,\xi}\ \tfrac{1}{2}\|\mathbf{w}\|^2 + C\, \Sigma_{i=1}{}^{N}(\xi_i + \xi_i^*) \tag{4}$$

subject to $y_i - f(\mathbf{x}_i) \le \varepsilon + \xi_i$ and $f(\mathbf{x}_i) - y_i \le \varepsilon + \xi_i^*$, so errors inside an ε-tube are ignored and C trades flatness against violations. We use the RBF kernel

$$K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma\|\mathbf{x} - \mathbf{x}'\|^2) \tag{5}$$

with $\gamma = 1/(p \cdot \text{Var}(\mathbf{X}))$ (scikit-learn `'scale'`). Following the paper's feature analysis, SVR uses only the top-4 inputs (S, u_b, n_p, T),

standardised in a pipeline. A 5-fold grid search over $C \in \{1, 10, 100, 500, 1000, 2048, 3000$ and $\varepsilon \in \{0.1, 0.5, 1, 2, 4$ selects $C = 100$, $\varepsilon = 4$ for both targets (the paper reports $C = 2048$, $\varepsilon = 4$). Models are scored with

$$R^2 = 1 - \Sigma_i(y_i - \hat{y}_i)^2 / \Sigma_i(y_i - \bar{y})^2 \tag{6}$$

4 Implementation Overview

`Dataset_Generator.py` writes the seeded `Dataset.csv`. `Models.py` preprocesses the data, trains the Random Forests (with an OOB sweep over $B \in \{10, \ldots, 400\}$), tunes the SVR via `GridSearchCV` on a `Pipeline(StandardScaler → SVR)`, computes 5-fold CV sensitivity curves for C and ε, and exports all figures to `Results/`. Stack: Python, NumPy, pandas, scikit-learn, seaborn, matplotlib. Run: `pip install -r requirements.txt`; `python3 Dataset_Generator.py`; `python3 Models.py`.

5 Results

Table 2. Test-set performance (R^2, 412 held-out samples).

Model	Inputs	Hyperparameters	R^2 (d_{50})	R^2 (σ^2)
Random Forest	all 8 features	$B = 400$ trees	0.840	**0.915**
SVR (RBF)	S, u_b, n_p, T	$C = 100$, $\varepsilon = 4$	**0.842**	0.903

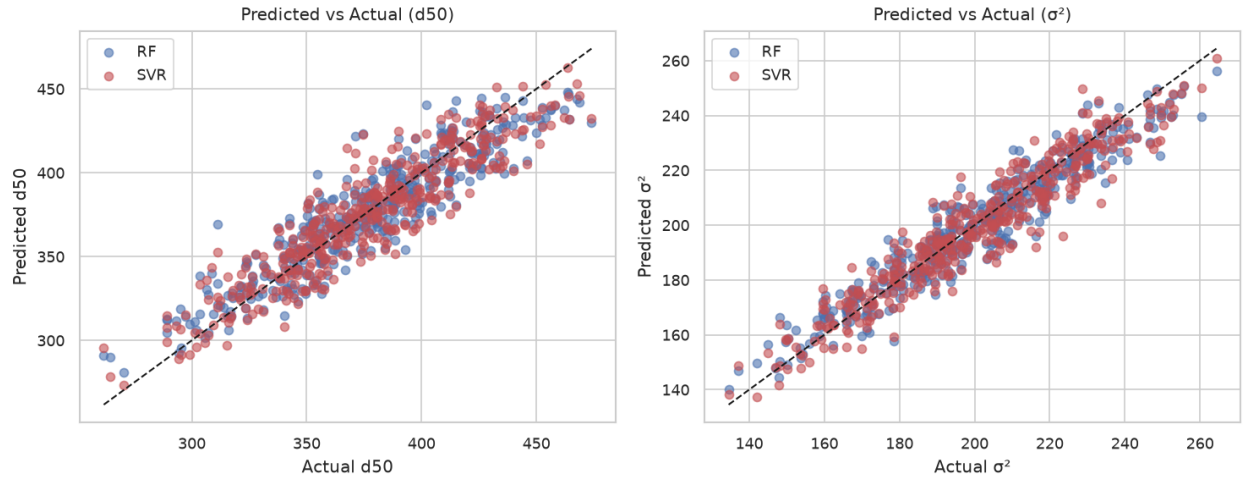


Figure 1. Predicted vs. actual d_{50} (left) and σ^2 (right) on the test set for Random Forest (blue) and SVR (red); the dashed line is the ideal $y = x$.

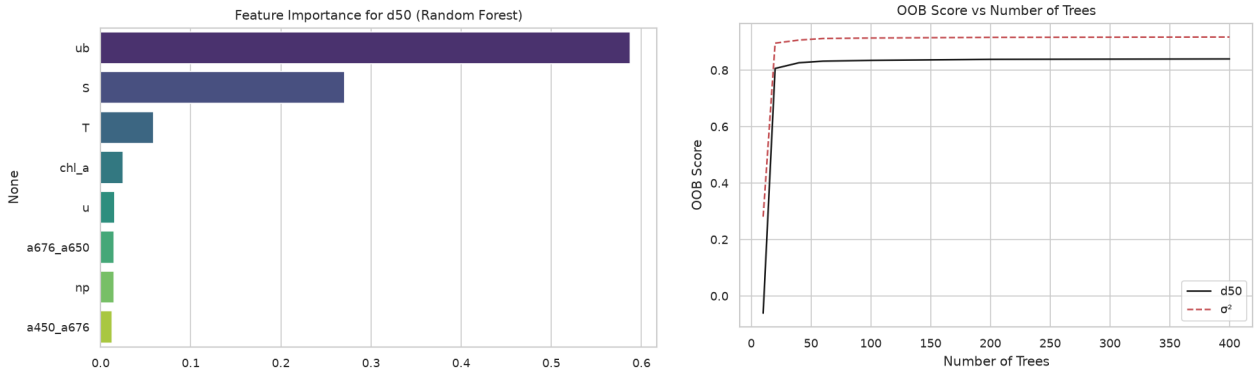


Figure 2. (a) Random Forest feature importance for d_{50}: wave-orbital velocity and salinity dominate. (b) OOB score versus number of trees for d_{50} (solid) and σ^2 (dashed).

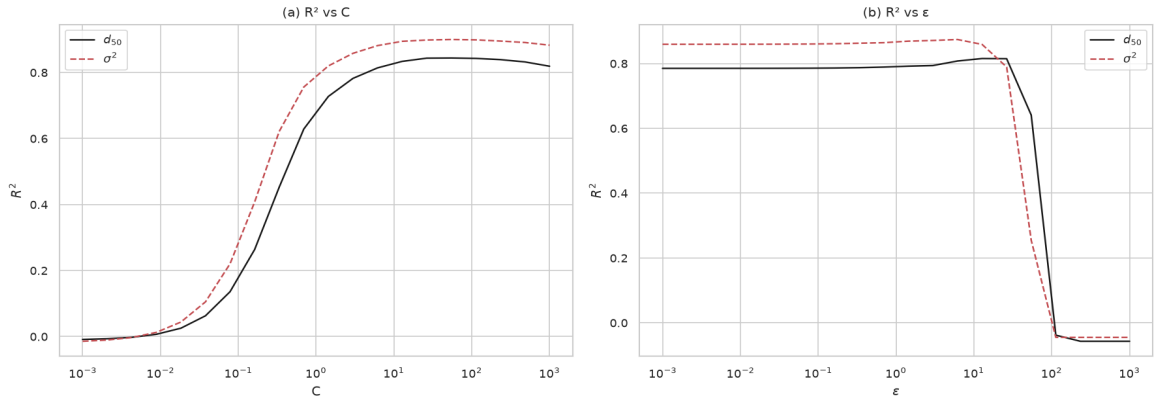


Figure 3. Five-fold CV R^2 of the SVR versus (a) the regularisation parameter C (ε fixed at 4) and (b) the tube width ε (C fixed at 2048), for d_{50} (solid) and σ^2 (dashed).

6 Discussion and Conclusions

Comparison. Both models capture the nonlinear flocculation structure. Random Forest is slightly better on σ^2 (0.915 vs. 0.903) and the two tie on d_{50} (0.840 vs. 0.842), echoing the paper's finding that neither algorithm clearly dominates; SVR gets there with half the inputs. σ^2 scores higher than d_{50} partly because the generator builds it from d_{50} (Eq. 2).

Features and tuning. Wave-orbital velocity (0.59) and salinity (0.27) carry about 86% of the importance for d_{50}, consistent with the paper's emphasis on S and u_b. On our synthetic data n_p ranks low (0.015) and chl-a fourth (0.025), a consequence of the generator's design rather than field truth; SVR keeps the paper's top-4 set for a faithful replication. In Fig. 3, R^2 rises sharply for $C \gtrsim 1$, plateaus beyond $C \approx 10$, and collapses once ε exceeds about 30, when the tube is wider than the spread of the targets.

Limitations. The data are synthetic, so the results validate the pipeline and modelling choices, not real-world accuracy; a single random split of autocorrelated hourly data may also be optimistic. Future work: real LISST/ac-9 data, time-blocked cross-validation, and gradient boosting or neural-network baselines.

Conclusion. Random Forest and SVR both predict d_{50} and σ^2 well ($R^2 \geq 0.84$ and ≥ 0.90), above the 0.75 benchmark, supporting data-driven models as an alternative to semi-empirical flocculation formulas.

References

[1] G. Egan, “Machine Learning for Predicting Sediment Particle Size Distributions,” Stanford CS229 Project Report, Autumn 2019.

[2] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[3] H. Drucker, C. Burges, L. Kaufman, A. Smola, V. Vapnik, “Support Vector Regression Machines,” *NIPS 9*, 1997.

[4] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *JMLR*, vol. 12, pp. 2825–2830, 2011.